Here's a partial implementation of the spec for you to add to the top of the file—not enough to pass, but enough to show that there's room for improvement in our specs:

11-matchers-included-in-rspec-expectations/exercises/phone_number_extractor_spec.rb

```
class PhoneNumberExtractor
  def self.extract_from(text, &block)
    # Look for patterns like (###) ###-####
    text.scan(/(d{3}) d{3}-d{4}/, &block)
  end
end
```

Run this file through RSpec. Now, take a look at the spec code. We had to do a lot of work to set up a separate collection for the yielded phone numbers, and then compare it afterward. Change this example to use a matcher better suited to how the extract_from method works. Notice how much simpler and clearer the spec is now.

Banner Year

The next two exercises will be in the same example group and will share the same implementation class. Let's start with the spec, which describes a fictional public company (named after a river) that had a good year. Add the following code to a new file:

11-matchers-included-in-rspec-expectations/exercises/public_company_spec.rb

```
RSpec.describe PublicCompany do
  let(:company) { PublicCompany.new('Nile', 10, 100_000) }

  it 'increases its market cap when it gets better than expected revenues' do
    before_market_cap = company.market_cap
    company.got_better_than_expected_revenues
    after_market_cap = company.market_cap

    expect(after_market_cap - before_market_cap).to be >= 50_000
  end
end
```

At the top of your file, add the following (not yet correct) implementation of the PublicCompany class:

11-matchers-included-in-rspec-expectations/exercises/public_company_spec.rb

```
PublicCompany = Struct.new(:name, :value_per_share, :share_count) do
  def got_better_than_expected_revenues
    self.value_per_share *= rand(1.05..1.10)
  end

  def market_cap
    @market_cap ||= value_per_share * share_count
  end
end
```